

Camada de Aplicação

Protocolos SMTP, servidores DNS e programação com sockets

Profa. Anelise Munaretto e Prof. Mauro Fonseca

DAINF - UTFPR

Material de Referência Bibliográfica

Kurose, J. F. & Ross, K. W. — *Redes de Computadores e a Internet: Uma Abordagem Top-Down*
https://gaia.cs.umass.edu/kurose_ross/

Como o E-mail Funciona: SMTP e Protocolos de Acesso

Como os Nomes São Resolvidos: DNS

Como Distribuir em Escala: P2P e BitTorrent

Como Programar a Rede: Sockets UDP e TCP

Síntese: A Inteligência na Borda

Como o E-mail Funciona: SMTP e Protocolos de Acesso

Correio Eletrônico: Três Componentes Principais

1. Agente de Usuário (UA)

- Interface do usuário (*composição, leitura*).
- **Clientes Nativos:** Thunderbird, Outlook.
- **Webmail:** Gmail, Outlook.com via browser.

2. Servidor de Correio

- **Caixa Postal:** Armazena mensagens recebidas.
- **Fila de Saída:** Mensagens aguardando envio.
- **Exemplos:** Postfix, Sendmail, Exchange.

3. Protocolos

- **SMTP (Porta 25/587):** Envio e repasse (*push*).
- **IMAP / POP3:** Resgate sob demanda (*pull*).

Arquitetura Fim-a-Fim do Correio Eletrônico



Servidores de Correio: Armazenamento e Filas

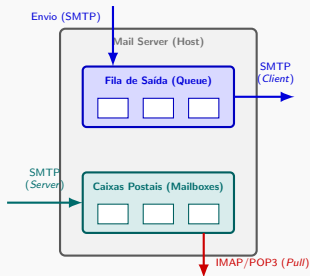
1. Fila de Mensagens (Message Queue)

- **Buffer de Saída:** Armazena e-mails enviados pelos clientes locais ou em trânsito de relay.
- **Tolerância a Falhas:** Se o destino estiver indisponível, agenda retransmissões automáticas (ex.: a cada 30 min por até 5 dias).

2. Caixa Postal (Mailbox)

- **Repositório Local:** Retém as mensagens recebidas em disco até a consulta pelo usuário.
- **Acesso Assíncrono:** O destinatário resgata seus e-mails sob demanda via **IMAP** ou **POP3**.

Arquitetura Interna do Mail Server



Servidores de Correio: A Dinâmica SMTP

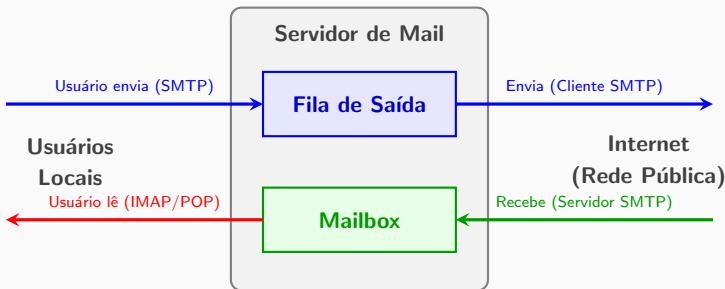
Para gerir filas e caixas de entrada, o servidor assume uma **Dupla Personalidade**

O Papel de Cliente

Ao enviar uma mensagem da sua **Fila de Saída** para a Internet, ele atua como **Cliente SMTP**

O Papel de Servidor

Ao receber uma mensagem da Internet para colocar na **Mailbox**, atua passivamente como **Servidor SMTP**



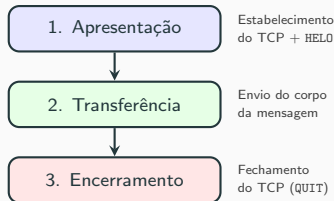
SMTP: Protocolo de Transferência

O SMTP (RFC 5321) é o protocolo padrão da Internet para a transferência confiável de correio eletrônico entre servidores

Características Técnicas

- **Transporte:** Utiliza conexões persistentes sobre **TCP (Porta 25)**
- **Entrega Direta:** O servidor de origem conecta-se *diretamente* ao servidor de destino (não há servidores intermediários de aplicação no caminho)
- **Formato:** Baseado em comandos de texto limpo (ASCII de 7 bits) e códigos numéricos de resposta (ex: 250 OK)

As Três Fases da Sessão



Cenário Prático: O Caminho do E-mail

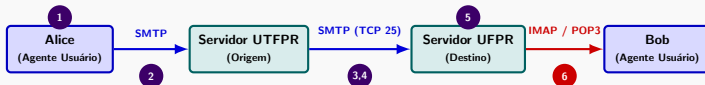
1. Da Origem à Fila (Alice)

1. **Composição:** Alice redige o e-mail no seu Agente de Usuário (UA).
2. **Envio Local (Push):** O UA envia ao servidor de correio da UTFPR via SMTP (entra na **Fila de Saída**).
3. **Sessão TCP:** O servidor da UTFPR abre conexão na porta 25 do servidor da UFPR.

2. Do Relay à Leitura (Bob)

4. **Transferência SMTP:** O servidor da Alice atua como cliente e transmite os dados.
5. **Armazenamento Local:** O servidor da UFPR recebe e armazena na **Caixa Postal (Mailbox)** do Bob.
6. **Resgate (Pull):** Bob abre o seu UA e baixa as mensagens via IMAP ou POP3.

Trilha Sequencial de Comunicação



Interação SMTP: O Diálogo em Texto Puro

A comunicação entre servidores utiliza comandos de texto simples. Abaixo, uma sessão real onde o servidor da Sorbonne entrega uma mensagem na UTFPR

Transcrição da Sessão (S: Servidor / C: Cliente)

```
S: 220 utfpr.edu.br
C: HELO sorbonne.fr
S: 250 Hello sorbonne.fr, pleased to meet you
C: MAIL FROM: <alice@sorbonne.fr>
S: 250 alice@sorbonne.fr... Sender ok
C: RCPT TO: <bob@utfpr.edu.br>
S: 250 bob@utfpr.edu.br ... Recipient ok
C: DATA
S: 354 Enter mail, end with '.' on a line by itself
C: Oi Bob, você já corrigiu as provas de Redes?
C: .      j- (Ponto isolado indica o Fim da Mensagem)
S: 250 Message accepted for delivery
C: QUIT
S: 221 utfpr.edu.br closing connection
```

- **Inspecção Manual:** Qualquer administrador pode usar o telnet (porta 25) para “falar” com o servidor
- **Códigos Numéricos:** Como no HTTP (200 OK), o SMTP responde com números (250, 354)

Comparação Técnica: SMTP vs. HTTP

Embora ambos utilizem comandos em texto puro e conexões TCP persistentes, suas filosofias de arquitetura são fundamentalmente opostas

HTTP: Protocolo de PULL (Puxar)

- O cliente toma a iniciativa de **buscar** a informação no servidor
- Cada objeto (HTML, imagem, vídeo) é transmitido na sua **própria mensagem** de resposta independente
- Lida nativamente com dados em formato binário

SMTP: Protocolo de PUSH (Empurrar)

- O servidor remetente toma a iniciativa de **entregar** o arquivo para o destino
- Múltiplos objetos (texto e anexos) são agrupados e enviados numa **única mensagem** (Multipart)
- **Não suporta** o envio de arquivos binários nativamente

A Restrição Histórica (ASCII de 7 bits)

O SMTP clássico exige que toda a comunicação (cabeçalhos e corpo da mensagem) seja estritamente em **ASCII de 7 bits**. Para o envio de arquivos binários (como PDFs, imagens) e caracteres com acentuação, tornou-se obrigatório o uso da extensão **MIME**, que codifica os arquivos em texto puro antes da transferência

Formato da Mensagem: Envelope vs. Conteúdo

Distinguir os comandos de roteamento (Envelope) do texto que o usuário lê (Mensagem) é essencial para a segurança do e-mail

1. O Envelope SMTP

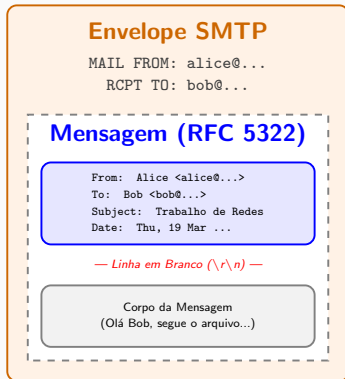
Controle usado **somente pelos servidores** para a entrega (criado no *handshake*)

- Comandos: MAIL FROM:, RCPT TO:
- O usuário final não enxerga estes dados

2. A Mensagem (RFC 5322)

A “carta” dentro do envelope, em duas partes:

- **Cabeçalho (Header):** Metadados lidos pelo cliente de e-mail (From:, To:, Subject:)
- **Corpo (Body):** O conteúdo redigido (em ASCII)
- **Regra de Ouro:** Cabeçalho e Corpo separados **obrigatoriamente** por uma linha em branco (CRLF)



Interação SMTP: O Envelope e a Mensagem

Veja como os comandos de roteamento (Envelope) se separam dos metadados da aplicação (Cabeçalho) durante a transferência:

Sessão Completa com Cabeçalhos (S: Servidor / C: Cliente)

```
S: 220 utfpr.edu.br
C: HELO sorbonne.fr
C: MAIL FROM: <alice@sorbonne.fr>    j- ENVELOPE (Roteamento)
S: 250 Sender ok
C: RCPT TO: <bob@utfpr.edu.br>    j- ENVELOPE (Roteamento)
S: 250 Recipient ok
C: DATA
S: 354 Enter mail, end with '.' on a line by itself
C: From: Alice <alice@sorbonne.fr>    j- CABEÇALHO (Mensagem)
C: To: Bob Professor <bob@utfpr.edu.br>    j- CABEÇALHO (Mensagem)
C: Subject: Dúvida sobre as provas    j- CABEÇALHO (Mensagem)
C:    j- (A obrigatória linha em branco!)
C: Oi Bob, você já corrigiu as provas de Redes?    j- CORPO (Mensagem)
C: .    j- (Fim da transferência)
S: 250 Message accepted for delivery
C: QUIT
```

MIME: Estendendo o SMTP (Exemplo Prático)

Como o SMTP clássico só entende texto ASCII (7 bits), o **MIME** converte arquivos binários (imagens, PDFs) e acentuação em texto puro antes do envio

Estrutura de uma Mensagem Multipartes (Raw Email)

```
From: alice@utfpr.edu.br
To: bob@utfpr.edu.br
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary='---Fronteira123'
```

(Linha em branco separa o cabeçalho global do corpo da mensagem)

```
---Fronteira123
Content-Type: text/plain; charset='utf-8'
```

Oi Bob, segue a logomarca que você pediu para a apresentação!

```
---Fronteira123
Content-Type: image/jpeg; name='logo.jpg'
Content-Transfer-Encoding: base64
```

/9j/4AAQSkZJRgABAQEASABIAAD/4QBMRXhpZgAATU0AKgAAAagA... (Bytes da imagem)
U29yYm9ubmUgVW5pdmVyc2l0w6k... (Mais texto convertido via Base64)

```
---Fronteira123-- j- (Fronteira final)
```

- **Base64:** Converte dados binários (8 bits) em caracteres ASCII seguros (A-Z, a-z, 0-9, +, /) aceitos pelo SMTP
- **Boundary:** *String* gerada pelo cliente de e-mail para separar o texto do anexo

Protocolos de Acesso: Recuperando o E-mail

1. POP3 (Post Office Protocol v3)

Filosofia: Baixar e Apagar.

Baixa para o disco local e apaga do servidor por padrão. Sem suporte nativo a múltiplos dispositivos.

2. IMAP (Internet Message Access Protocol)

Filosofia: Sincronização Remota.

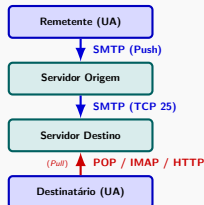
Mantém mensagens e pastas no servidor. Sincroniza o estado de leitura em múltiplos dispositivos.

3. HTTP / HTTPS (Webmail)

Filosofia: Interface Web.

O navegador atua como Agente de Usuário, acessando o servidor de correio via requisições Web.

Push (SMTP) vs. Pull (Acesso)



Protocolo POP3: Máquina de Estados Simples

Embora seja um protocolo legado, o POP3 é um excelente exemplo didático de comunicação baseada em texto. Ele opera em três fases principais:

1. Autorização

Identificação e login em texto puro (inseguro sem TLS)

- user: nome do usuário
- pass: senha

2. Transação e Atualização

O usuário manipula as mensagens

- list: lista IDs e tamanhos
- retr: recupera mensagem
- dele: marca para exclusão
- quit: encerra a sessão (o servidor apaga as mensagens marcadas)

Sessão POP3 (S: Servidor / C: Cliente)

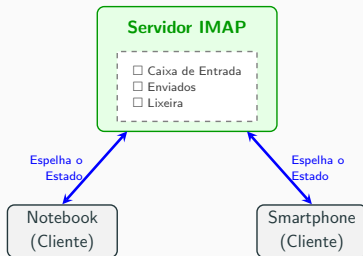
```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user logged on
C: list      |← (Fase de Transação)
S: 1 498
S: 2 912
S: .
C: retr 1
S: <conteúdo da mensagem 1, vários bytes>
C: dele 1    |← (Marca para apagar)
C: quit      |← (Fase de Atualização)
S: +OK POP3 server signing off
```

Protocolos de Acesso: O Domínio do IMAP

Com a necessidade de acessar a mesma conta em múltiplos locais (computador, celular, tablet), o protocolo **IMAP** tornou-se o padrão absoluto para leitura e gestão

Características do IMAP

- **Gestão Centralizada:** Todas as mensagens permanecem armazenadas no servidor. O cliente atua apenas como um “espelho”
- **Organização Remota:** Permite ao usuário criar, renomear e apagar pastas diretamente no servidor
- **Protocolo com Estado (Stateful):** O servidor memoriza o status de cada mensagem (ex: já lida, não lida, respondida, apagada)
- **Eficiência:** Permite baixar apenas os cabeçalhos das mensagens, adiando o download de anexos pesados para quando o usuário solicitar



Webmail: O Navegador como Agente

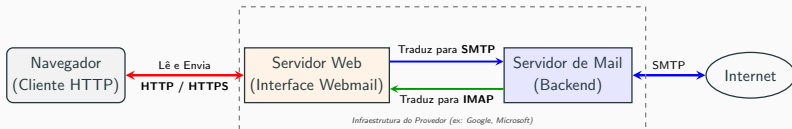
No ecossistema de Webmail (Gmail, Outlook.com, etc.), o usuário não instala um cliente de e-mail tradicional. O **Navegador Web** assume o papel de *User Agent*

A Mudança no Transporte

A comunicação entre o usuário e o seu provedor de e-mail deixa de usar SMTP ou IMAP diretamente, passando a usar exclusivamente o **HTTP/HTTPS**

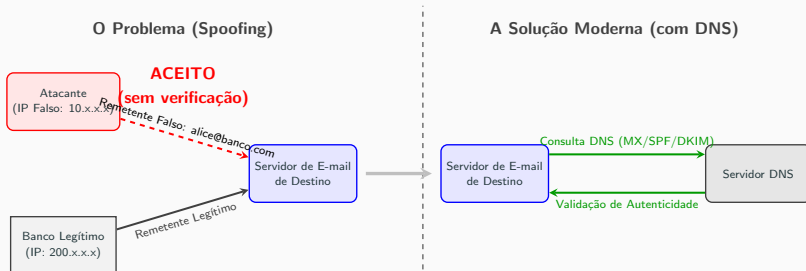
O Servidor como Tradutor

O servidor web atua como um *proxy*. Ele recebe os cliques do usuário na página web e os traduz em comandos **SMTP** (para enviar) ou **IMAP** (para gerir pastas) no *backend*



Segurança em E-mail: A Ameaça do Spoofing

O protocolo SMTP original não verifica se o endereço IP que está enviando a mensagem realmente tem autorização para usar aquele domínio, permitindo fraudes



ALERTA DE SEGURANÇA

É a infraestrutura do **DNS** que permite ao servidor de destino consultar registros públicos (como SPF) para confirmar se o IP emissor tem delegação oficial para enviar e-mails em nome daquele domínio. **Siglas:** SPF (Sender Policy Framework), DKIM (DomainKeys Identified Mail) e DMARC (Domain-based Message Authentication Reporting and Conformance)

Autenticação Moderna: A Trindade da Segurança

Hoje, um e-mail só atinge a caixa de entrada se passar por um rigoroso processo de validação cruzada, apoiado pela infraestrutura pública do DNS

1. SPF (Origem)

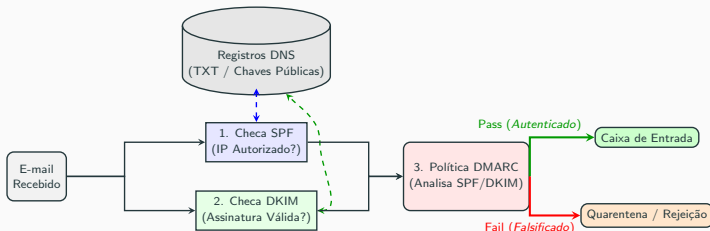
O que faz: Valida o IP
O destino consulta o registro TXT no DNS para verificar se o IP remetente está na lista autorizada do domínio (evita spoofing direto)

2. DKIM (Integridade)

O que faz: Assinatura Digital
O remetente assina o e-mail. O destino baixa a chave pública do DNS para garantir que o conteúdo não foi adulterado no caminho

3. DMARC (Política)

O que faz: O Grande Juiz
Usa SPF + DKIM para ditar a regra: observar (*None*), reter (*Quarantine*) ou bloquear (*Reject*)



Como os Nomes São Resolvidos: DNS

DNS: Por que não usamos apenas o IP?

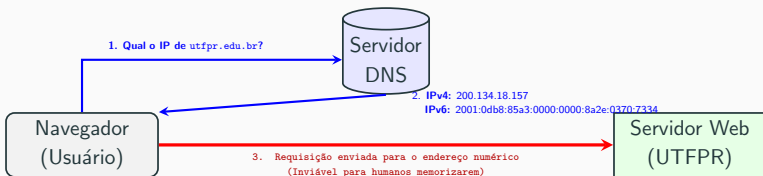
O DNS resolve o conflito entre a memorização humana e a precisão das máquinas. Imagine ter que digitar o endereço IPv6 abaixo para cada acesso:

O Desafio da Identificação

- **Mnemônico:** `www.utfpr.edu.br`
- **IPv4 (32 bits):** `200.134.18.157`
- **IPv6 (128 bits):**
`2001:db8:85a3::8a2e:370:7334`

O Papel do DNS

- **Abstração:** Oculta a complexidade da infraestrutura de endereçamento
- **Flexibilidade:** Permite mudar o servidor (IP) sem alterar o nome que o usuário conhece



DNS: Além da Tradução e o Desafio da Escala

O DNS não é apenas um “tradutor”; ele é uma camada estratégica que oferece serviços essenciais para a resiliência da rede

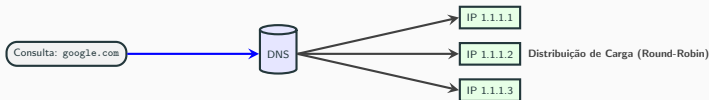
Serviços Agregados

- **Host Aliasing:** Um nome canônico (*canonical name*) pode ter vários apelidos curtos
- **Mail Server Aliasing:** Registros **MX** garantem que o e-mail chegue ao destino correto, independente do servidor web
- **Distribuição de Carga:** Um único nome mapeia para uma lista de IPs, distribuindo o tráfego entre réplicas

Por que não centralizar?

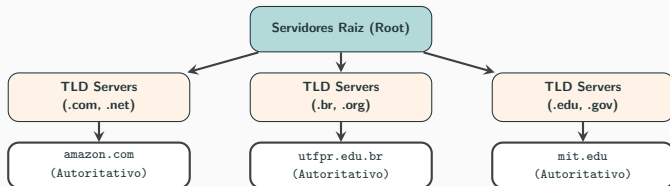
Uma base única seria catastrófica:

- **Ponto Único de Falha:** Se o servidor cai, a Internet global para
- **Volume de Tráfego:** Bilhões de consultas simultâneas saturariam qualquer enlace
- **Latência Geográfica:** Consultar um servidor em outro continente é inviável
- **Manutenção:** Impossível gerir bilhões de registros num único local



Arquitetura Hierárquica: A Árvore do DNS

Para garantir escalabilidade e gestão descentralizada, o DNS organiza-se em uma estrutura em árvore, onde a autoridade é delegada do topo para a base



O Fluxo de Resolução

Para resolver `www.utfpr.edu.br`:

1. Consulta o **Raiz** para achar o **.br**
2. Consulta o **TLD(Top-Level Domain) (.br)** para achar a **UTFPR**
3. Consulta o **Autoritativo** da UTFPR para obter o IP final

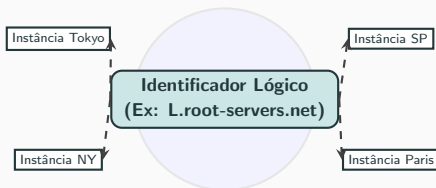
Responsabilidades

- **Root:** Sabe onde estão os TLDs
- **TLD:** Gere domínios de topo
- **Autoritativo:** Detém o mapeamento real Nome → IP da organização

Os Servidores Raiz: A Base da Pirâmide

Os servidores raiz são o primeiro ponto de contato quando um servidor local não possui a resposta em *cache*

- **Função:** Direcionam a consulta para o servidor TLD (Top-Level Domain) correspondente (.com, .br, etc.)
- **Infraestrutura Lógica vs. Física:**
 - Existem apenas **13 identificadores lógicos** (letras A até M)
 - **Anycast:** Cada letra é, na verdade, uma rede de mais de **1.600 instâncias** espalhadas pelo mundo para garantir resiliência



Resiliência Anycast

Se uma instância falha ou sofre um ataque DDoS, o protocolo de roteamento BGP direciona o tráfego automaticamente para a próxima instância mais próxima, sem interrupção do serviço

TLD e Servidores Autoritativos: A Gestão de Nomes

A hierarquia DNS distribui a autoridade para que nenhuma entidade gerencie todos os nomes do mundo

1. Servidores TLD

(Top-Level Domain) Responsáveis pelo sufixo do domínio

- **TLD (Genéricos):** .com, .net, .edu
- **TLD (Países):** .br, .ar, .jp

Exemplo: No Brasil, o **Registro.br** opera o .br

2. Servidores Autoritativos

Detêm os registros oficiais da organização

- Fornecem o mapeamento **Nome** → **IP** final
- Essencial para qualquer instituição com presença na rede

Exemplo: Servidores da **UTFPR** que resolvem `portal.utfpr.edu.br`



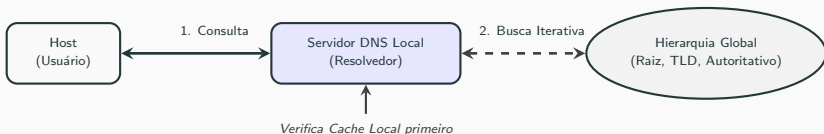
Cloud DNS

Empresas podem delegar seus servidores autoritativos para provedores (AWS Route 53, Cloudflare) para maior resiliência contra ataques

Servidor de Nomes Local (Local DNS Server)

Embora não faça parte da hierarquia oficial (Raiz/TLD), ele é o “Resolvedor Recursivo” que faz o trabalho pesado para o usuário

- **Onde ele está?** Presente em cada ISP (Ex: Copel, Vivo), campus da **UTFPR** ou rede corporativa
- **Papel de Intermediário (Proxy):**
 - O host (PC/Celular) faz uma consulta simples ao servidor DNS local
 - Se a resposta não estiver no **Cache**, o servidor DNS local percorre a hierarquia global por conta própria
- **Vantagem:** Reduz a latência para nomes populares e economiza banda no enlace de saída da instituição



Configuração Automática

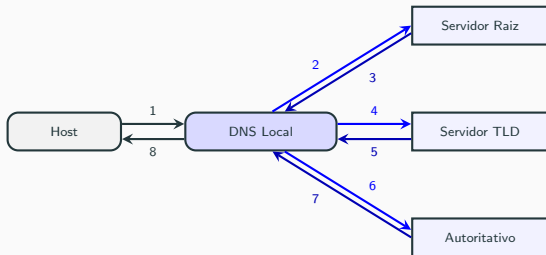
Na maioria dos casos, o endereço do Servidor DNS Local é fornecido automaticamente ao seu dispositivo via protocolo **DHCP** no momento em que você se conecta à rede

Resolução de Nomes: Consulta Iterativa

Na consulta iterativa, o servidor consultado não resolve o nome; ele responde com uma **referência** para o próximo servidor na hierarquia

Fluxo de Resolução:

1. Host consulta o **DNS Local**
2. DNS Local pergunta ao **Raiz**
3. Raiz aponta o IP do **TLD**
4. DNS Local pergunta ao **TLD**
5. TLD aponta o **Autoritativo**
6. DNS Local pergunta ao **Autoritativo**
7. Autoritativo entrega o **IP Final**
8. DNS Local responde ao Host



Responsabilidade

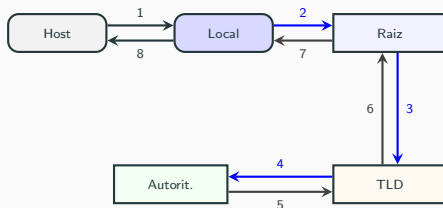
O **DNS Local** é quem trabalha. Os servidores de topo apenas "indicam o caminho", economizando recursos de processamento globais

Resolução de Nomes: Consulta Recursiva

Na consulta recursiva, o servidor consultado **assume a dívida** de encontrar a resposta, agindo como cliente do próximo nível da hierarquia

A Dinâmica da “Corrente”:

- O servidor **não indica o caminho**; ele percorre o caminho pelo usuário
- Transfere a carga de processamento para o topo da árvore
- **Vantagem:** O cliente (Host) faz apenas uma pergunta simples
- **Risco:** Facilita ataques de negação de serviço (DDoS) e sobrecarrega os Servidores Raiz



Política de Segurança

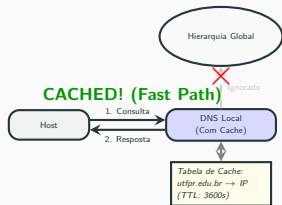
Devido ao alto consumo de recursos e riscos de segurança, a maioria dos **Servidores Raiz** e **TLDs** brasileiros e mundiais desativam o suporte a consultas recursivas

DNS Caching: O Herói da Performance

O cache é o que impede a sobrecarga da infraestrutura global, permitindo que consultas repetidas sejam resolvidas instantaneamente no nível local

Aprendizado e Memória

- **O Atalho:** Uma vez que o DNS Local aprende um mapeamento (Nome → IP), ele o guarda para consultas futuras
- **Hierarquia Protegida:** TLDs (.com, .br) quase sempre em cache ⇒ os **Servidores Raiz** raramente são consultados



O Parâmetro TTL

Time To Live (Tempo de Vida):

- É o “prazo de validade” definido pelo dono do domínio
- Após o TTL expirar, a entrada é descartada para garantir a **consistência** (evitar IPs antigos)

Nota: O comando `ipconfig /displaydns` (Windows) ou `sudo dscacheutil -cachedump` (macOS) permite visualizar o cache DNS local da sua máquina

Registros DNS: Resource Records (RR)

A base de dados do DNS gerencia diferentes tipos de recursos através de tuplas estruturadas

Formato do Registro: (Name, Value, Type, TTL)

O significado dos campos **Name** e **Value** é ditado pelo **Type**

Tipo	Nome (Name)	Valor (Value)	Exemplo (UTFPR)
A	Hostname	Endereço IPv4	portal → 200.134.x.x
NS	Domínio	Hostname do Servidor	utfpr.edu.br → ns1...
CNAME	Apelido (Alias)	Nome Canônico (Real)	www → portal...
MX	Domínio	Servidor de E-mail	utfpr.edu.br → google...

Dica: CNAME

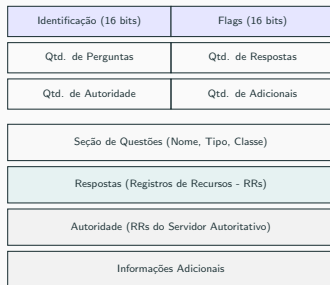
Permite que o servidor mude de IP ou nome real sem que o endereço `www` conhecido pelo usuário mude

Registros TXT

Usados para segurança (SPF/DKIM). Guardam strings de texto que provam a identidade do remetente

Protocolo DNS: Formato das Mensagens

O DNS utiliza um formato de mensagem único e simplificado para consultas e respostas, operando sobre **UDP** (porta 53) para minimizar a latência



Campos do Cabeçalho

- **ID:** Número de 16 bits que "casa" a resposta com a pergunta enviada
- **Contadores:** Indicam quantos registros existem em cada seção abaixo

Flags Importantes

- **QR:** 0 para consulta, 1 para resposta
- **AA:** Resposta veio de um servidor autoritativo?
- **RD/RA:** Recursão Desejada / Disponível

Registro de Domínios e Propagação

Para colocar um serviço online (ex: `utfpr.edu.br`) é necessário registrar o nome em uma entidade oficial, como o **Registro.br**

Passo 1: A Entidade de Registro

Você fornece o nome e o IP dos seus servidores autoritativos (Primário e Secundário)

Passo 2: Inserção no TLD

O *Registrar* insere os registros (Name, Value, Type, TTL) no servidor **TLD** (.br):

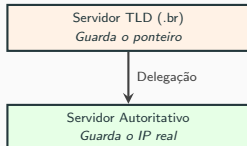
- (`utfpr.edu.br`, `dns1.utfpr.edu.br`, NS, 86400)
- (`dns1.utfpr.edu.br`, `200.x.x.x`, A, 86400)

**O Registro A é essencial para quebrar a dependência circular*

Passo 3: Configuração Local

No seu servidor autoritativo, você cria os mapeamentos finais:

- (`www.utfpr.edu.br`, `200.y.y.y`, A, 86400)
- (`utfpr.edu.br`, `mail.utfpr.edu.br`, MX, 86400)



Segurança: DNS como Alvo Crítico

DNS Poisoning: Insere registros falsos no *cache* para redirecionar usuários a sites maliciosos

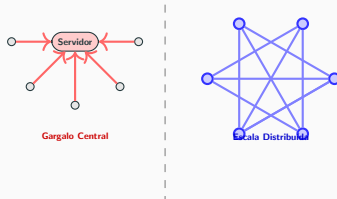
Como Distribuir em Escala: P2P e BitTorrent

Arquitetura P2P: O Paradigma Descentralizado

Ao contrário do modelo centralizado, na arquitetura P2P a inteligência, a banda de rede e o armazenamento são fornecidos pelos próprios usuários

A Mágica da Auto-escalabilidade

Cada novo nó (*peer*) que entra na rede traz uma nova requisição (aumenta a demanda), mas também traz **capacidade de upload** (aumenta a oferta). O sistema escala naturalmente



O Preço: Dinamismo Extremo

- **Intermitência (Churn):** Os nós ligam-se e desligam-se da rede sem aviso prévio
- **IPs Dinâmicos:** Exige protocolos complexos de rastreamento para saber quem tem qual arquivo no momento

Exemplos Clássicos:

- **BitTorrent:** Distribuição de arquivos pesados
- **Blockchain:** Descentralização de confiança (Bitcoin)

Matemática da Distribuição: O Gargalo Cliente-Servidor

O Problema: Quanto tempo leva para enviar um arquivo de tamanho F para N clientes simultaneamente?

Definição das Variáveis

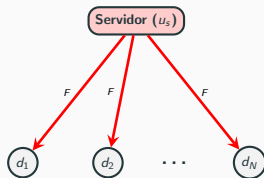
- u_s : Banda de *upload* do servidor
- d_{min} : Banda de *download* do cliente mais lento

Tempo de Distribuição (D_{c-s})

$$D_{c-s} \geq \max \left\{ \frac{N \cdot F}{u_s}, \frac{F}{d_{min}} \right\}$$

Interpretando a Equação:

- **Termo 1 (Servidor):** O servidor é obrigado a enviar N cópias inteiras. O tempo consumido no seu link é $N \cdot F / u_s$
- **Termo 2 (Cliente):** O cliente com a pior internet dita o limite inferior absoluto de tempo (F / d_{min})



A Grande Conclusão

Como a variável N multiplica o tempo de upload do servidor, o tempo de distribuição cresce de forma **linear e infinita**. O servidor torna-se um gargalo físico

Matemática da Distribuição: A Escalabilidade P2P

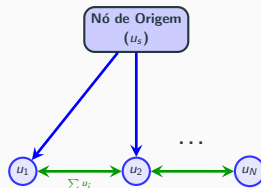
A Solução: No modelo P2P, a arquitetura ganha escala porque cada cliente que faz *download* também ajuda no *upload*, injetando banda na rede

Tempo de Distribuição (D_{p2p})

$$D_{p2p} \geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{N \cdot F}{u_s + \sum_{i=1}^N u_i} \right\}$$

O Tempo será o pior gargalo entre os 3 limites:

- **Termo 1 (A Origem):** O nó de origem (*seeder*) precisa fazer *upload* de pelo menos uma cópia para a malha (F/u_s)
- **Termo 2 (O Elo Fraco):** A distribuição não acaba até que o *peer* com a pior internet consiga baixar tudo (F/d_{min})
- **Termo 3 (Capacidade Global):** O sistema precisa transferir $N \cdot F$ bits. A "força" que empurra esses bits é o nó de origem mais todos os *peers* ($u_s + \sum u_i$)



A Grande Vantagem

Repare no denominador do 3º termo: à medida que N cresce (aumentando a demanda), a soma $\sum u_i$ também cresce (aumentando a oferta). O tempo total estabiliza

Exemplo Prático: A Matemática em Ação

Vamos simular a distribuição de uma imagem ISO de Sistema Operacional para provar o colapso do modelo centralizado em larga escala

Parâmetros do Cenário

- **Arquivo (F):** 15 Gbits (≈ 1.87 GB)
- **Upload do Servidor (u_s):** 30 Mbps
- **Download do Cliente (d_{min}):** 2 Mbps
- **Upload do Cliente (u_i):** 0.3 Mbps

O Detalhe Oculto

O cliente tem 2 Mbps para baixar, mas só colabora com **0.3 Mbps** (300 Kbps) de upload no P2P. Será que essa pequena ajuda de cada um salva a rede?

Tempo Total de Distribuição para todos os N clientes

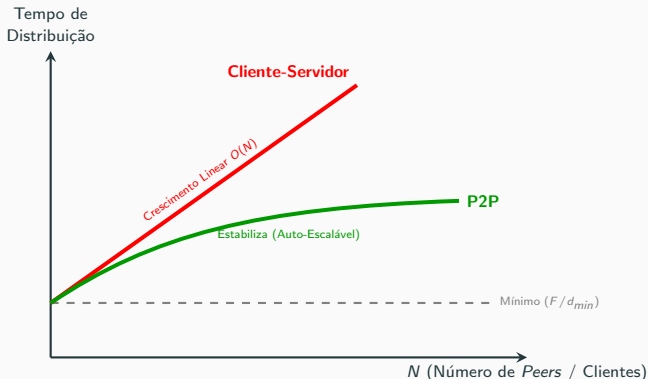
Arquitetura	$N = 10$ clientes	$N = 100$ clientes	$N = 1.000$ clientes
Cliente-Servidor	2,1 horas	13,8 horas	138,8 horas (Quase 6 dias!)
Peer-to-Peer (P2P)	2,1 horas	6,9 horas	12,6 horas

Conclusão do Exemplo

- Com $N = 10$: O gargalo é a internet de 2 Mbps do próprio usuário (F/d_{min}). Ambas empatam em 2,1 horas
- Com $N = 1000$: O servidor C-S colapsa tentando enviar 1.000 cópias em um link de 30 Mbps (leva 6 dias). No P2P, a soma dos uploads de 0.3 Mbps de todos os usuários gera uma "super banda" de 300 Mbps, terminando tudo em meio dia

Gráfico de Desempenho: C-S vs. P2P

A diferença de escalabilidade torna-se inegável quando observamos o comportamento matemático das duas arquiteturas em larga escala ($N \rightarrow \infty$)



O Veredito da Escalabilidade

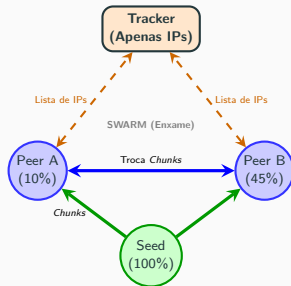
Em redes massivas, a arquitetura P2P é a única solução viável. É por isso que o mercado utiliza essa lógica para distribuir atualizações gigantescas que colapsariam servidores tradicionais (ex: *Updates* do Windows 10/11, *patches* de jogos pesados)

BitTorrent: Terminologia e Anatomia do Enxame

O BitTorrent é o protocolo de distribuição de arquivos mais bem-sucedido da história, desenhado para maximizar o uso da banda de *upload* dos usuários

O Dicionário do Protocolo

- **Torrent (Metadados):** Arquivo estático que contém os *hashes* criptográficos de verificação e o endereço do *Tracker*
- **Tracker (Rastreador):** O “Páginas Amarelas”. Um servidor que anota quem entra e quem sai da rede. **Não armazena o arquivo**
- **Chunks (Pedaços):** O arquivo alvo é fatiado em blocos minúsculos (tipicamente 256 KB)
- **Swarm (Enxame):** O grupo de todos os nós (*peers*) ativamente trocando *chunks* daquele arquivo



A Regra de Ouro

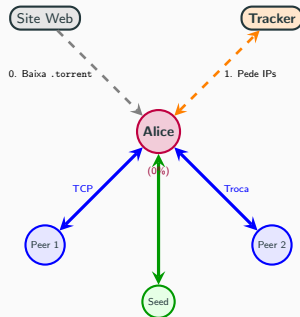
Enquanto você faz o *download* de um *chunk*, atua obrigatoriamente como servidor, fazendo o *upload* desse mesmo *chunk* para outro *peer*

Dinâmica do Enxame: O Caso da Alice

Como um novo nó entra na rede? Vamos acompanhar o ciclo de vida da Alice

O Ciclo de Vida (Passo a Passo)

1. **O .torrent:** Alice baixa o arquivo .torrent (ou *magnet*) de um site Web comum. Ele contém o mapa (*hashes*) e a URL do Tracker
2. **Registro:** Ela contata o **Tracker** listado no arquivo para avisar que chegou e recebe a lista de IPs dos vizinhos
3. **Troca (Swarming):**
 - Alice chega "de mãos vazias" (0 pedaços)
 - Abre conexões TCP diretas e pede pedaços, enquanto faz *upload* do que já tem



Churn e Comportamento Final

Ao completar 100%, Alice faz uma escolha:

- **Egoísmo (Leecher):** Desconecta e foge
- **Altruísmo (Seeder):** Continua conectada distribuindo

Dinâmica do Enxame: A Inteligência da Alice

Como a Alice decide **quais** pedaços pedir e para **quem** enviar? O sucesso do BitTorrent depende de duas lógicas geniais para evitar o colapso da rede

1. O que pedir? (Rarest First)

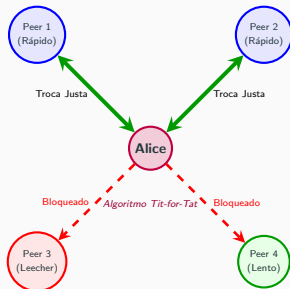
Ao verificar o que seus vizinhos têm, Alice solicita primeiro os pedaços que são **mais raros** no enxame

- **Objetivo:** Garantir que os pedaços difíceis se multipliquem rapidamente, evitando que o arquivo “morra” se o dono original (Seed) se desconectar

2. Para quem enviar? (Tit-for-Tat)

Alice tem uma banda de *upload* limitada. Como ela distribui isso?

- **A Regra:** “Olho por olho”. Alice mede constantemente a taxa de *download* que recebe dos vizinhos
- Ela só envia dados (*Unchoke*) para os 4 vizinhos que estão lhe enviando dados **mais rápido**
- **Resultado:** *Leechers* (sugadores) que não enviam nada são asfixiados e bloqueados (*Choked*) pela rede



Resumo Comparativo: Cliente-Servidor vs. P2P

Para encerrar o tópico de arquiteturas de aplicação, aqui está o quadro resumo com as diferenças fundamentais entre os dois modelos

Característica	Cliente-Servidor (C-S)	Peer-to-Peer (P2P)
Ponto de Falha	Único: Se o servidor cai, a rede inteira para	Distribuído: A saída de um nó não derruba a rede
Escalabilidade	Baixa: O servidor torna-se o gargalo (crescimento linear)	Alta: A capacidade cresce com a demanda (auto-escalável)
Desempenho (Alta Carga)	Degrada rapidamente	Estabiliza ou melhora
Gerenciamento	Simples: Tudo é controlado de forma centralizada	Complexo: Exige gestão de IPs dinâmicos e nós intermitentes
Segurança e Confiança	Alta (controle de acesso centralizado)	Baixa (nós maliciosos podem distribuir dados falsos)
Exemplos Clássicos	Web (HTTP), E-mail (SMTP), Streaming (Netflix)	BitTorrent, Blockchain, VoIP inicial (Skype)

O Paradoxo da Escolha

Apesar do P2P ser superior em distribuição em massa, a maioria dos serviços (Netflix, Spotify) usa **Cliente-Servidor** + **CDNs** para controlar direitos autorais e qualidade

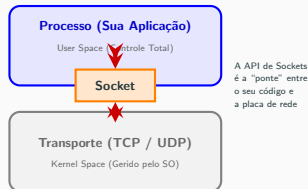
Como Programar a Rede: Sockets UDP e TCP

Programação com Sockets: O Ponto de Entrada

O objetivo agora é entender como construir aplicações (cliente/servidor) que se comunicam através da rede usando a API do Sistema Operacional

O que é um Socket?

É a “porta” lógica através da qual o processo da aplicação envia e recebe mensagens da rede. É a fronteira entre o que você programa e o que o SO gerencia



O Limite de Controle (API)

- **Acima do Socket (Aplicação):** O programador tem controle total (o que enviar, quando enviar, como estruturar a mensagem)
- **Abaixo do Socket (Transporte):** O controle é delegado ao Sistema Operacional. O programador apenas escolhe o protocolo (TCP ou UDP) e ajusta parâmetros básicos (ex: tamanho do *buffer*)

Tipos de Sockets: Escolhendo o Transporte

A primeira grande decisão do programador ao criar um *socket* é definir o protocolo de transporte, que ditará o comportamento da rede

1. Socket UDP (SOCK_DGRAM)

- **Datagramas:** Envio de pacotes isolados (como “cartas” no correio)
- **Não confiável:** Não garante a entrega, nem a ordem dos dados
- **Vantagem:** Muito rápido, sem o peso (*overhead*) de manter uma conexão



2. Socket TCP (SOCK_STREAM)

- **Fluxo de Bytes:** Leitura e escrita contínuas
- **Confiável:** Garante a entrega e a ordem exata dos dados
- **Conexão:** Cria um “tubo” virtual (*pipe*) entre o cliente e o servidor



O Nosso “Hello World” das Redes (Exemplo Prático)

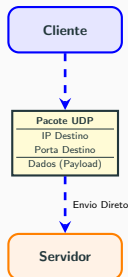
1. O **Cliente** lê uma frase digitada no teclado e envia pelo *socket*
2. O **Servidor** recebe a frase e converte tudo para **LETRAS MAIÚSCULAS**
3. O **Servidor** devolve a frase modificada e o cliente imprime na tela

Programação com Sockets UDP: Sem Conexão

O UDP (Datagrama) caracteriza-se pela ausência total de uma fase de estabelecimento de conexão (*handshaking*) antes do envio dos dados

A Dinâmica do Envio e Recepção

- **Endereçamento Explícito:** Como não há um “tubo” aberto, o remetente deve carimbar o **IP** e a **Porta de destino** em *cada* pacote enviado
- **A Resposta:** O servidor extrai o IP e a porta de origem de dentro do pacote que acabou de chegar para saber para onde devolver a resposta



Analogia:
Envio de carta
pelos Correios

Performance vs. Confiabilidade

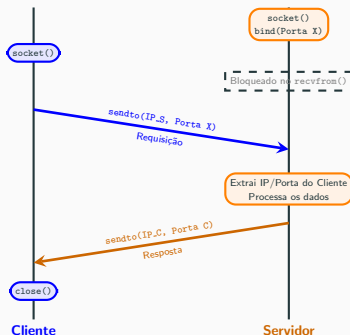
- **A Vantagem:** Menor latência inicial (os dados saem imediatamente)
- **O Custo:** Pacotes podem se perder na rede, chegar duplicados ou fora da ordem original

Fluxo de Interação via UDP: A API de Sockets

Como não há conexão prévia, o fluxo de métodos chamados na programação de um *socket* UDP é direto e orientado a pacotes (*datagramas*)

O Lado do Servidor

1. `socket()`: Cria o ponto de entrada lógico
2. `bind()`: Associa o *socket* a uma porta fixa (ex: 12000) e fica escutando a rede
3. `recvfrom()`: Recebe o pacote, extraíndo os dados e o endereço (IP e Porta) do remetente
4. `sendto()`: Envia a resposta de volta usando o endereço extraído



O Lado do Cliente

1. `socket()`: Cria o ponto de entrada local
2. `sendto()`: Envia os dados, carimbando explicitamente o IP e a Porta do servidor
3. `recvfrom()`: Aguarda a resposta chegar
4. `close()`: Encerra o *socket*

Cliente UDP em Python: Sockets e Camadas

cliente_udp.py

```
from socket import *

# Destino: Tupla (IP, Porta)
server = ('127.0.0.1', 12000)

# Socket UDP (IPv4, Datagrama)
clientSocket = socket(AF_INET, SOCK_DGRAM)

# Leitura e Envio
msg = input('Digite a mensagem: ')
clientSocket.sendto(msg.encode(), server)

# Recepção e Fechamento
resp, addr = clientSocket.recvfrom(2048)
print('Servidor:', resp.decode())

clientSocket.close()
```

O Papel de .encode() e .decode()

- **Modelo OSI:** Representa a **Camada de Apresentação (Camada 6)**, responsável pela conversão e sintaxe dos dados.
- **Pilha TCP/IP:** Executado diretamente na lógica da **Aplicação**.
- **Transporte (UDP):** Trafega unicamente sequências brutas de **bytes** (octetos).

Parâmetros de Socket

- **AF_INET:** Família de endereços IPv4.
- **SOCK_DGRAM:** Protocolo UDP (sem conexão prévia).

Servidor UDP em Python

servidor_udp.py

```
from socket import *

# Cria o socket UDP (IPv4, Datagrama)
serverSocket = socket(AF_INET, SOCK_DGRAM)

# Fixa a porta ('' escuta em todas as interfaces de rede)
serverSocket.bind('', 12000)
print('Servidor pronto para receber pacotes...')

while True:
    # Bloqueia e aguarda pacote. Salva dados e IP/Porta de origem
    msg, addr = serverSocket.recvfrom(2048)

    # Regra de negocio: converte texto para maiusculas
    mod_msg = msg.decode().upper()

    # Envia resposta de volta usando o endereco capturado
    serverSocket.sendto(mod_msg.encode(), addr)
```

Segredo: recvfrom entrega dados + tupla do remetente (addr) $\Rightarrow$ o sendto sabe para onde responder

Servidor UDP em Python: Estrutura e Escuta

servidor_udp.py

```
from socket import *

# Cria o socket UDP (IPv4, Datagrama)
serverSocket = socket(AF_INET, SOCK_DGRAM)

# Bind: escuta em todas as interfaces na porta 12000
serverSocket.bind('', 12000)
print('Servidor pronto para receber pacotes...')

while True:
    # Bloqueia até receber pacote; captura dados e addr
    msg, addr = serverSocket.recvfrom(2048)

    # Processamento (Apresentação / Aplicação)
    mod_msg = msg.decode().upper()

    # Resposta direcionada usando a tupla do remetente
    serverSocket.sendto(mod_msg.encode(), addr)
```

Endereçamento Dinâmico

- **Tupla addr:** Retornada por `recvfrom`, contém (IP, Porta) de origem do remetente.
- **Modelo Sem Estado (Stateless):** O servidor não mantém sessão aberta; usa `addr` diretamente no `sendto`.

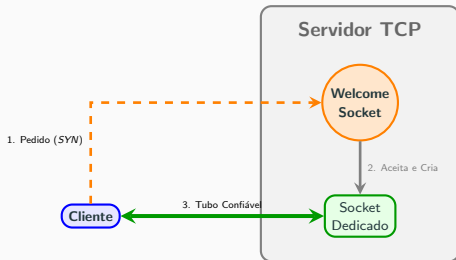
Configuração do Socket

- `bind('', 12000)`: A string vazia '' instrui o SO a escutar em **todas as interfaces** de rede ativas.
- `recvfrom(2048)`: Define o limite do buffer de recepção em 2048 bytes.

Programação com Sockets TCP

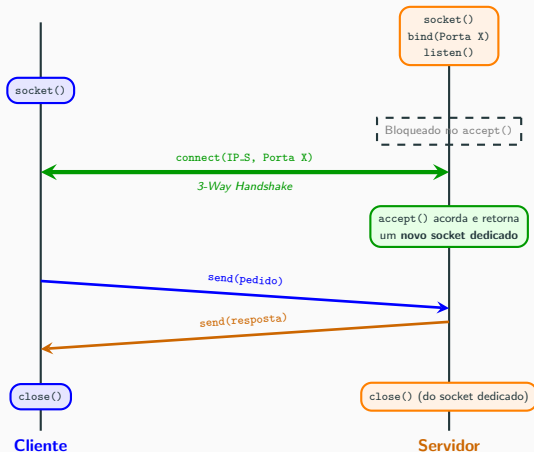
Ao contrário do UDP, o TCP exige uma conexão prévia (*handshake*), garantindo um fluxo de dados rigoroso e confiável

- **Welcome Socket:** O servidor fica “escutando” a rede aguardando novos contatos (“Porta de Boas-Vindas”)
- **Handshake:** O cliente solicita a conexão. O Sistema Operacional resolve o *3-way handshake*
- **Socket Dedicado:** Ao aceitar a conexão, o servidor cria um **novo socket** exclusivo para se comunicar com aquele cliente
- **Reliable Pipe:** A comunicação ocorre como em um “tubo” virtual (os bytes chegam em ordem e sem perdas)



Fluxo de Interação via TCP: A API de Sockets

O TCP adiciona etapas cruciais antes da troca de dados: o servidor precisa entrar em estado de “escuta” e o cliente deve forçar a conexão



Servidor TCP: Welcoming vs. Connection Socket

Diferente do UDP, o TCP separa o ato de **receber** um cliente do ato de **conversar** com ele

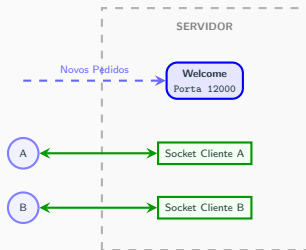
- **Socket de Boas-vindas (Welcoming):**

- Criado uma única vez pelo servidor
- Fica em `listen()` na porta oficial (ex: 80)
- Sua única função é "atender o interfone"

- **Socket de Conexão (Connection):**

- Criado pelo SO a cada `accept()` bem-sucedido
- É um **novo objeto** dedicado a um único cliente
- Permite ao servidor atender múltiplos clientes em paralelo

- **Identificação:** O SO diferencia os sockets pela **quádrupla**: (IP/Porta Origem) + (IP/Porta Destino)



Resumo: O *Welcome Socket* é a porta de entrada da loja; o *Connection Socket* é o vendedor exclusivo que acompanha cada cliente

Cliente TCP em Python: Conexão e Fluxo de Bytes

cliente_tcp.py

```
from socket import *

# Destino: Tupla (IP, Porta)
server = ('127.0.0.1', 12000)

# 1. Socket TCP (IPv4, Fluxo Contínuo)
client = socket(AF_INET, SOCK_STREAM)

# 2. Executa o TCP Handshake (3-Way)
client.connect(server)

# 3. Envia bytes pelo fluxo estabelecido
msg = input('Digite a mensagem: ')
client.send(msg.encode())

# 4. Recebe resposta do buffer e encerra
resp = client.recv(1024)
print('Servidor:', resp.decode())

client.close()
```

Estabelecimento da Conexão

- **SOCK_STREAM**: Define o protocolo orientado a conexão (**TCP**).
- **connect(server)**: Dispara o *3-Way Handshake* (SYN → SYN-ACK → ACK) antes do envio de dados.

send() vs. sendto()

- **Sem Endereçamento Explícito**: O socket já está associado à conexão do host remoto; não se passa tupla de destino.
- **Fluxo Contínuo (Byte Stream)**: O TCP não preserva fronteiras de mensagens (*framing* fica a cargo da aplicação).

Servidor TCP em Python: Escuta e Conexão Dedicada

servidor_tcp.py

```
from socket import *

# 1. Socket de boas-vindas (Welcoming Socket)
ss = socket(AF_INET, SOCK_STREAM)
ss.bind(('', 12000))
ss.listen(1) # Tamanho da fila de conexões pendentes
print('Servidor pronto...')

while True:
    # 2. Bloqueia até handshake e gera novo socket dedicado
    conn, addr = ss.accept()

    # 3. Troca de dados pelo fluxo dedicado
    msg = conn.recv(1024).decode()
    conn.send(msg.upper().encode())

    # 4. Encerra apenas a conexão com o cliente atual
    conn.close()
```

Dois Tipos de Sockets TCP

- **Socket de Boas-Vindas (ss):** Permanece fixo na porta 12000 apenas escutando requisições de conexão (`listen`).
- **Socket de Conexão (conn):** Criado dinamicamente por `accept()` exclusivamente para a troca de dados com o cliente aceito.

Parâmetro `listen(1)`

- Define o tamanho da fila (*backlog*) de conexões no SO aguardando a chamada de `accept()`.
- `conn.close()`: Encerra o socket do cliente, mas `ss` segue ouvindo novos clientes.

Design de Protocolos: Quatro Decisões Fundamentais

1. Manutenção de Estado (State)

- **Com Estado (Stateful):** O servidor retém o contexto e histórico entre interações (ex.: *IMAP, FTP*).
- **Sem Estado (Stateless):** Cada transação é independente e atômica (ex.: *HTTP, DNS*).

2. Garantia de Transporte

- **Confiável (TCP):** Entrega estrita, ordenada e com controle de fluxo/congestionamento (ex.: *HTTP, SMTP*).
- **Tolerante a Perdas (UDP):** Foco em mínima latência e ausência de handshake (ex.: *DNS, VoIP*).

3. Canal de Controle (Signaling)

- **In-Band:** Mensagens de controle e dados trafegam no mesmo fluxo/conexão (ex.: *HTTP, SMTP*).
- **Out-of-Band:** Separação física ou lógica de canais: um para comandos e outro para dados (ex.: *FTP*).

4. Paradigma de Distribuição

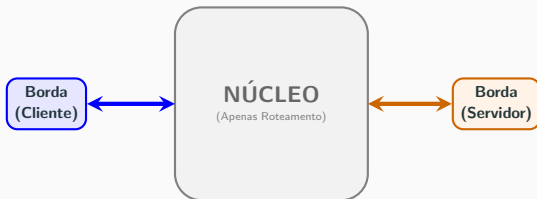
- **Cliente-Servidor:** Centralizado; limitado pela capacidade de enlace do servidor (ex.: *Web, E-mail*).
- **Peer-to-Peer (P2P):** Descentralizado e autoescalável; nós atuam como clientes e servidores (ex.: *BitTorrent*).

Síntese: A Inteligência na Borda

Resumo: O Princípio da Inteligência na Borda

A grande sacada da Internet: **complexidade nos extremos, simplicidade no meio**

- **Núcleo Simples:** Roteadores apenas repassam pacotes, sem entender o conteúdo
- **Borda Inteligente:** Toda a lógica (protocolos, erros, interface) roda exclusivamente nos *hosts*
- **Escalabilidade:** Lançar um novo app global não exige alterar a rede física



Conclusão: Dominar os protocolos de aplicação é ter controle total sobre a inteligência do sistema

E-mail e SMTP

- **SMTP (Push, TCP 25)**: entrega direta em 3 fases (apresentação, transferência, encerramento)
- **Mensagem**: envelope (MAIL FROM / RCPT TO) + cabeçalho e corpo; **MIME** para binários
- **Leitura (Pull)**: POP3 (transferir e apagar) vs. IMAP (sincronização) vs. Webmail (HTTP)
- **Segurança**: SPF + DKIM + DMARC contra *spoofing*

DNS

- **Hierarquia**: raiz (*anycast*) → TLD → autoritativo; DNS local com *cache/TTL*
- **Consultas**: iterativa (referências) vs. recursiva (assume a resolução)
- **Registros**: A, NS, CNAME, MX e TXT (SPF/DKIM); mensagens sobre UDP 53

P2P e BitTorrent

- **Matemática**: D_{c-s} cresce com N (gargalo no servidor); D_{p2p} **estabiliza** (auto-escala)
- **BitTorrent**: *tracker*, *chunks* (256 KB), *swarm*; *rarest-first* e *tit-for-tat*

Sockets (Python)

- **UDP (SOCK_DGRAM)**: sem conexão; *sendto/recvfrom* com IP+porta por pacote
- **TCP (SOCK_STREAM)**: conexão confiável; *connect/send/recv*; *welcome* vs. *connection socket*
- **Decisões de projeto**: estado, confiabilidade, canal *in/out-of-band*, arquitetura